



JC4003 Natural Language Processing

Lecture 3: Context-Free Grammars and Parsing

Dr Xiao Li

xiao.li@abdn.ac.uk

Dr Ruizhe Li

ruizhe.li@abdn.ac.uk

Dr Siwei Liu

siwei.liu@abdn.ac.uk

Context-Free Grammars and Parsing

Grammar study has a long history. The Indian grammarian [Pāṇini \(पाणिनि\)](#) described the grammar of Sanskrit between the 7th and 4th centuries BCE in his famous work "अष्टाध्यायी" (Aṣṭādhyāyī, The Eight Books). It summarises Sanskrit grammar into 3,959 grammatical rules. It is the [first known work](#) in the world to codify the grammar of a natural language.

The word "syntax" comes from the Greek word "sýntaxis," which means "arrangement" or "putting together." It refers to the way words are organised in sentences.

Outline

Context-Free Grammars (CFGs)

Parse Trees

CKY Parsing Algorithm

Ambiguity in Parsing and Probabilistic CKY Parsing

Limitation of CFG/PCFG

Context-Free Grammars (CFGs)

Context-Free Grammars

A Context-Free Grammar (CFGs) is a system that uses specific rules to show how sentences in a language are constructed. These rules define how elements of a sentence, like nouns and verbs, are combined (i.e. how non-terminal symbols can be replaced by sequences of terminal symbols and other non-terminals).

NB: The term "context-free" indicates that these rules does not depend on the surrounding symbols (i.e., the context).

Non-terminals, Terminals, & Rules

Non-terminals: These are **abstract symbols** that represent groups of words, e.g. sentences (S), noun phrases (NP), or verb phrases (VP). They help structure the sentence but don't appear in the actual text.

Terminals: These are the **actual words** in the language, e.g. 'cat', 'sees', or 'the'. They are the final output that we read in sentences.

Production Rules: These **rules show how to build sentences by replacing non-terminals** with terminals or other non-terminals, e.g. a rule might say that a sentence (S) consists of a noun phrase (NP) followed by a verb phrase (VP), denoted by $S \rightarrow NP VP$

Non-terminals, Terminals, & Rules (cont.)

Here's a simple example of how CFGs work:

Non-terminals:

NP (Noun Phrase),

Det (Determiner),

ProperNoun

Nominal

Noun

Terminals:

a,

the,

flight,

Production Rules:

NP $\rightarrow$ Det Nominal

NP $\rightarrow$ ProperNoun

Nominal $\rightarrow$ Noun | Nominal Noun

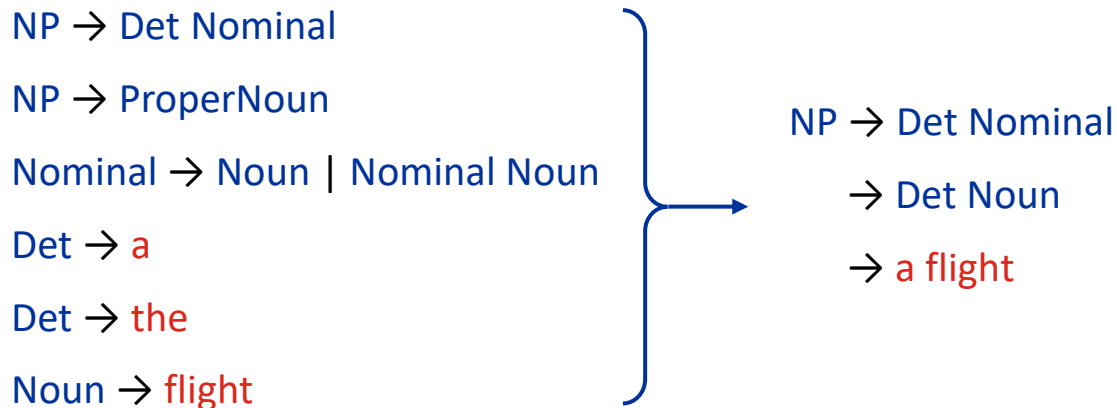
Det $\rightarrow$ a

Det $\rightarrow$ the

Noun $\rightarrow$ flight

Generating Sentences

As a device for generating sentences and as a device for assigning a structure to a given sentence. Viewing a CFG as a generator, we can read the $\rightarrow$ arrow as “rewrite the symbol on the left with the string of symbols on the right”. E.g.:



Generating Sentences (cont.)

The formal language defined by a CFG is the set of strings that are derivable from the designated **start symbol**. Each grammar must have one designated start symbol, which is often called **S**, representing "sentence".

With adding a few rules, a sentence can consist of a noun phrase followed by a verb phrase.

$S \rightarrow \text{NP VP}$ I prefer a morning flight
 $VP \rightarrow \text{Verb NP}$ prefer a morning flight

Or, the verb may be followed by a noun phrase and a **prepositional phrase**:

$VP \rightarrow \text{Verb NP PP}$ leave Boston in the morning

A Simple CFG Example: $\mathcal{L}_0$

$\mathcal{L}_0$

Noun → flights | flight | breeze | trip | morning
Verb → is | prefer | like | need | want | fly | do
Adjective → cheapest | non-stop | first | latest |
other | direct
Pronoun → me | I | you | it
Proper-Noun Alaska | Baltimore | Los Angeles |
→ Chicago | United | American
the | a | an | this | these | that
Determiner → from | to | on | near | in
Preposition → and | or | but
Conjunction →

Lexicon for $\mathcal{L}_0$

S → NP VP
NP → Pronoun
| Proper-Noun
| Det Nominal
Nominal → Nominal Noun
| Noun
VP → Verb
| Verb NP
| Verb NP PP
| Verb PP
PP → Preposition NP

Grammar Rules for $\mathcal{L}_0$

I + want a morning flight
I
Los Angeles
a + flight
morning + flight
flights
do
want + a flight
leave + Boston + in the morning
leaving + on Thursday
from + Los Angeles

Examples

Formal Language

A CFG like $\mathcal{L}_0$ specifies a **formal language**.

Sentences that can be generated by the grammar are considered "**grammatical**" in this formal language. If a sentence can't be generated by the grammar, it's considered "**ungrammatical**" in that context.

This clear distinction between what is and isn't allowed by the grammar is typical of formal languages. **However, this approach is quite simplified** compared to how natural languages like English work, where context often determines whether a sentence is acceptable.

In linguistics, using formal languages to model natural languages is known as **generative grammar**, as it defines the language by the sentences that can be "generated" according to the grammar rules.

Formal Definition of CFG

A context-free grammar G (for the formal language $\mathcal{L}_G$) is defined by four parameters: N , Σ , R , S . That is: $G = \{N, \Sigma, R, S\}$

- N : a set of non-terminal symbols (or variables)
- Σ : a set of terminal symbols (disjoint from N)
- R : a set of rules or productions, each of the form $A \rightarrow \beta$,
where A is a non-terminal,
 β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$
- S : a designated start symbol and a member of N

Context-Free Grammars (cont.)

CFGs are useful in NLP for:

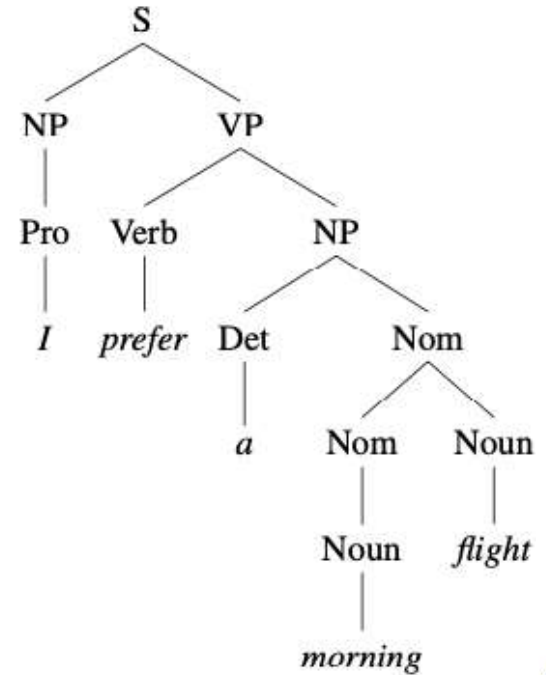
- **Parsing**: Analysing the structure of sentences to understand their grammar.
- **Grammar Checking**: Identifying mistakes in sentences by checking them against grammar rules.
- **Sentence Generation**: Creating new sentences that are grammatically correct.

Parse Tree

Parse Tree

A **parse tree**, also known as a **syntax tree**, is a tree-like diagram that represents the syntactic structure of a sentence according to a given grammar.

A parse tree is derived from a CFG and represents the **syntactic structure of a sentence** according to the rules defined in the CFG.



A parse tree for
"I prefer a morning flight"

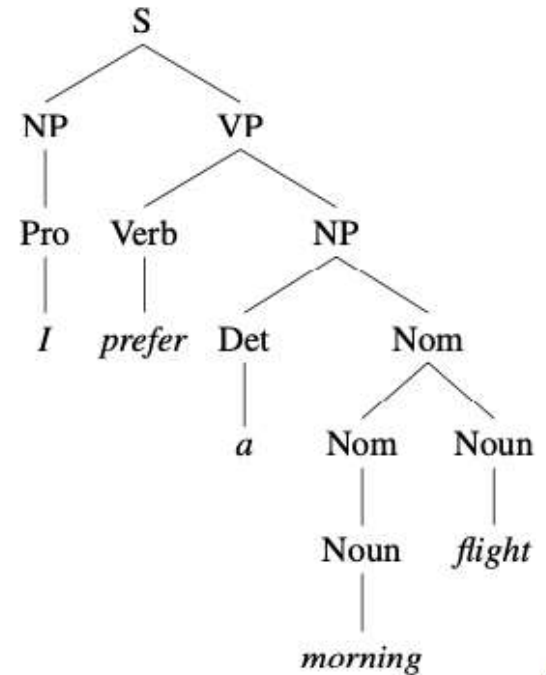
Parse Tree

Internal nodes represent non-terminal symbols denote syntactic categories and structural rules.

Leaf nodes are terminal symbols, i.e. the actual words in the sentence.

The **branches** in the tree connect the nodes, illustrating how the sentence is divided into its constituent parts.

The **root node**, usually labelled as 'S' (Sentence), represents the entire sentence.

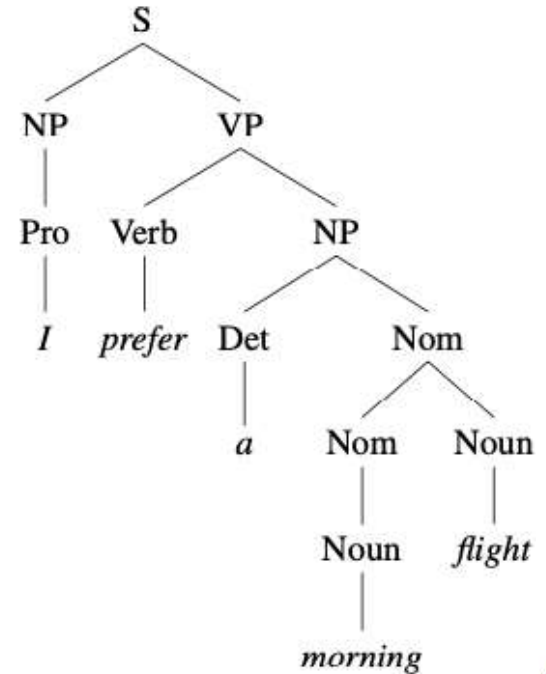


A parse tree for
"I prefer a morning flight"

Parse Tree

We can also represent a parse tree in a more compact format called **bracketed notation**; here is the bracketed representation of the parse tree:

[S [NP [Pro I]] [VP [V prefer] [NP [Det a] [Nom [N morning] [Nom [N flight]]]]]]]



A parse tree for
"I prefer a morning flight"

CKY Parsing

Chomsky Normal Form

Chomsky Normal Form (CNF) is a specific way of structuring CFGs that makes them easier to work with in computational processes, particularly in parsing algorithms like the CYK (Cocke-Younger-Kasami) algorithm.

In CNF, all rules of a CFG must adhere to one of the following formats:

- A non-terminal produces exactly two non-terminals
e.g. $S \rightarrow NP VP$ ~~$S \rightarrow NP$~~
- A non-terminal produces a single terminal symbol
e.g. $Noun \rightarrow book \mid flight$ ~~$Noun \rightarrow flight ticket$~~

Conversion to CNF

Every CFG can convert to CNF. To do so, the following steps are typically involved:

- **Remove Unit Productions:** Eliminate rules where a non-terminal produces another single non-terminal.

E.g. $A \rightarrow B, B \rightarrow \text{flight} \Rightarrow A \rightarrow \text{flight}$

- **Convert Remaining Rules:** Break down rules that have 3 or more non-terminals on the right-hand side into multiple rules that fit the CNF structure.

E.g. $A \rightarrow B C D \Rightarrow A \rightarrow B X, X \rightarrow C D$

- **Eliminate Useless Symbols:** Remove symbols that produce an empty string, or do not appear in any derivation of the start symbol.

CKY Algorithm

The CKY (Cocke-Kasami-Younger) algorithm is a well-known parsing algorithm used for syntactic analysis of sentences according to a given grammar in [CNF](#).

The CKY algorithm is a dynamic programming method that [systematically fills in a parse table](#) to determine whether a given sentence can be generated by the grammar and, if so, to construct a [corresponding parse tree](#).

CKY Algorithm Example

We use CKY to parse the sentence "Book the flight through Aberdeen", with the grammar:

Verb $\rightarrow$ book
Noun $\rightarrow$ book | flight
Proper-Noun $\rightarrow$ Aberdeen
Det $\rightarrow$ the
Preposition $\rightarrow$ through

CFG Lexicon

S $\rightarrow$ VP
VP $\rightarrow$ Verb NP
VP $\rightarrow$ Verb NP PP
NP $\rightarrow$ Det Nominal
NP $\rightarrow$ Proper-Noun
Nominal $\rightarrow$ Noun
Nominal $\rightarrow$ Nominal PP
PP $\rightarrow$ Preposition NP

CFG Syntax

CKY Algorithm Example (cont.)

We use CKY to parse the sentence "Book the flight through Aberdeen", with the grammar:

Verb $\rightarrow$ book
Nominal $\rightarrow$ book | flight
NP $\rightarrow$ Aberdeen
Det $\rightarrow$ the
Preposition $\rightarrow$ through

CNF Lexicon

~~S $\rightarrow$ VP~~
S $\rightarrow$ Verb NP
S $\rightarrow$ S PP
NP $\rightarrow$ Det Nominal
~~NP $\rightarrow$ Proper Noun~~
~~Nominal $\rightarrow$ Noun~~
Nominal $\rightarrow$ Nominal PP
PP $\rightarrow$ Preposition NP

CNF Syntax

CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

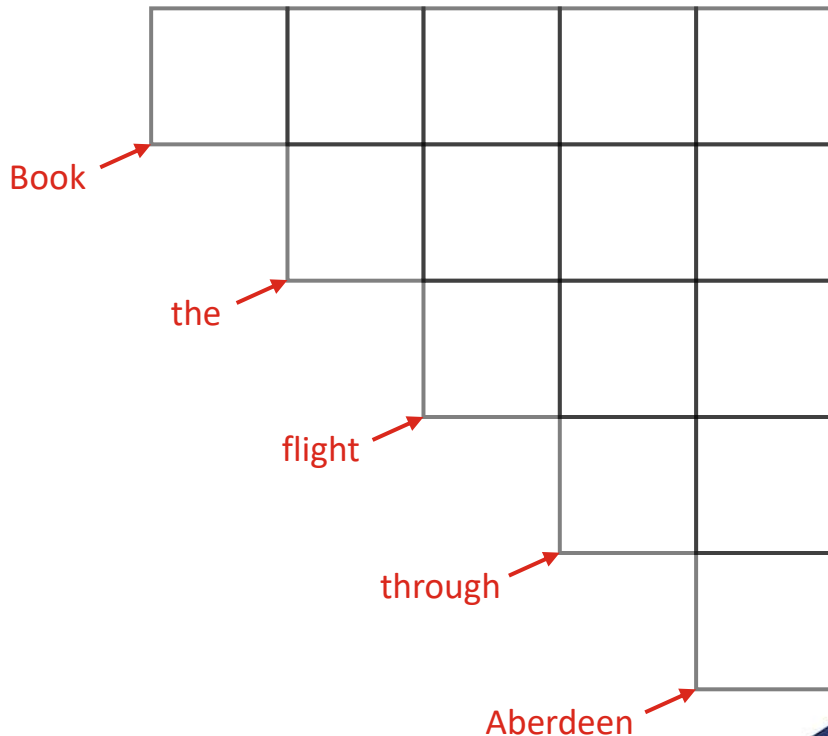
Verb → **book**

Nominal → **book** | **flight**

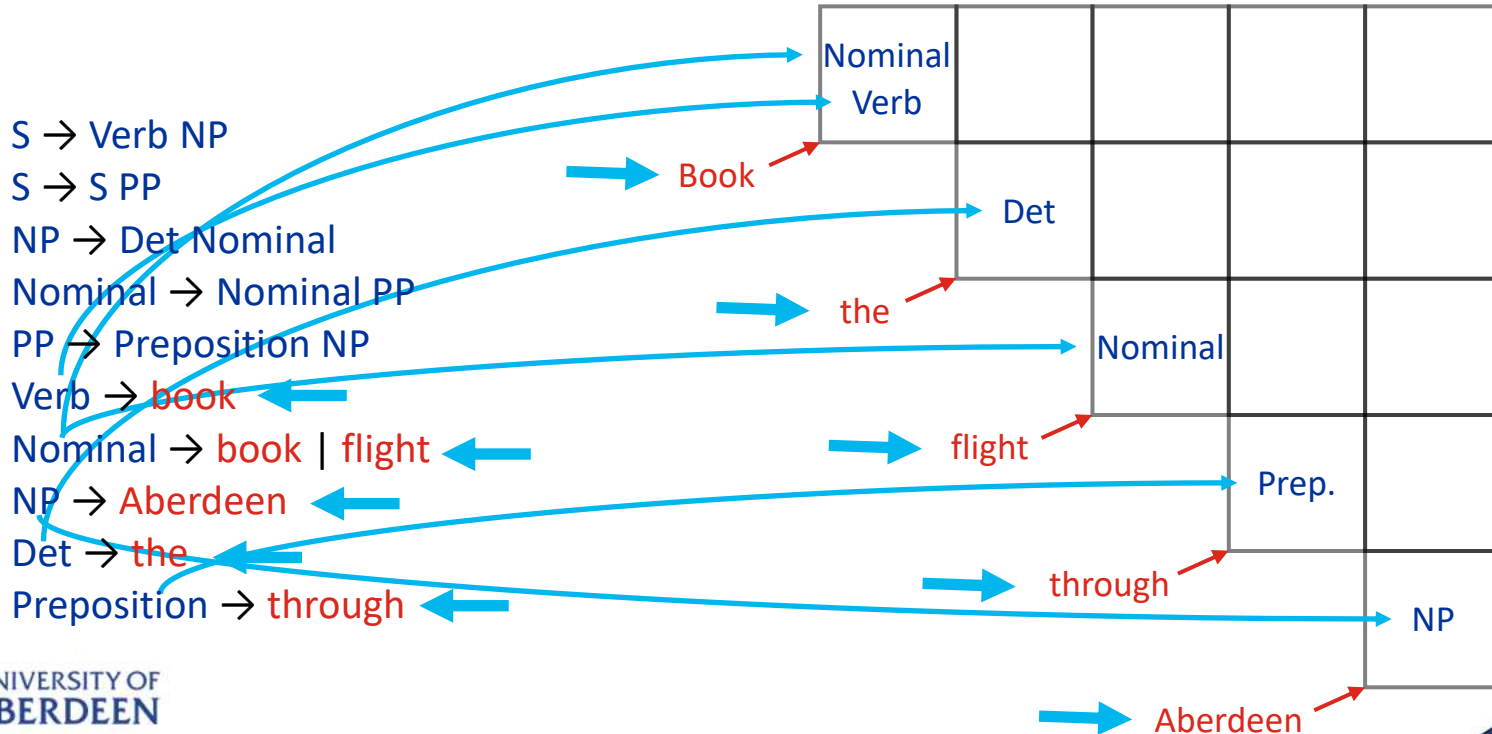
NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$$S \rightarrow S \text{ PP}$$

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

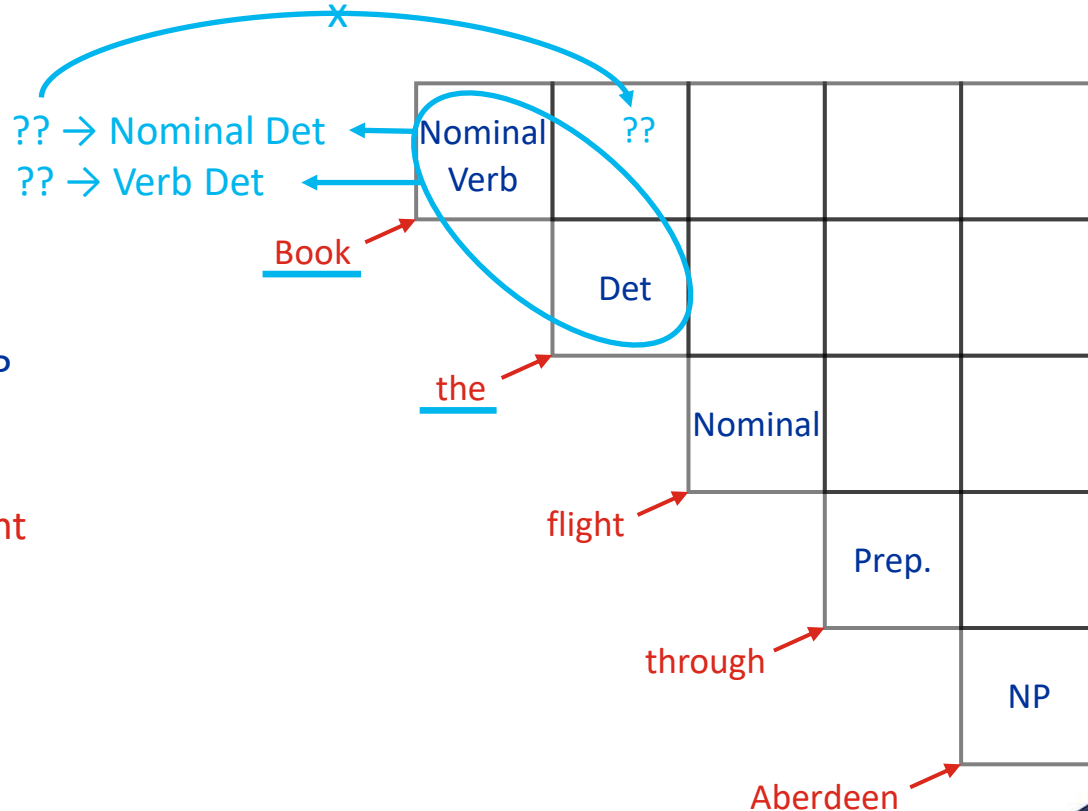
Verb → book

Nominal $\rightarrow$ book | flight

NP → Aberdeen

Det $\rightarrow$ the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal → NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

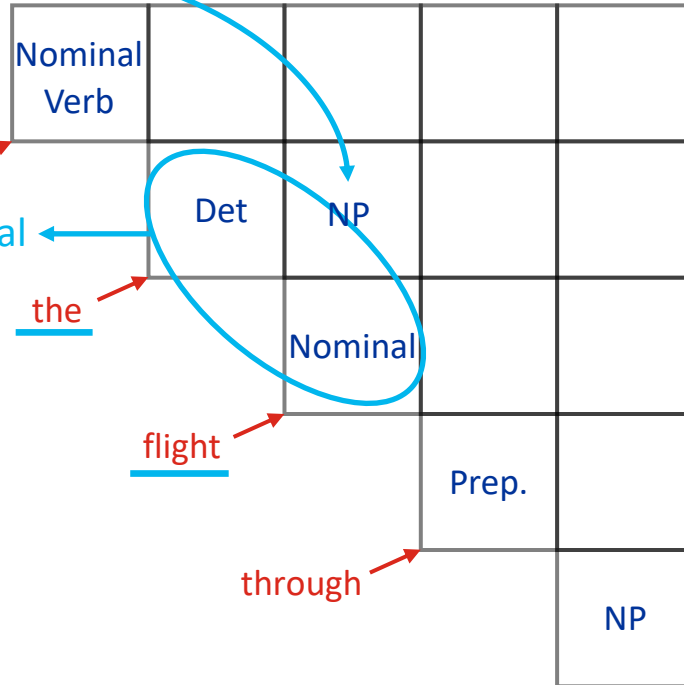
Verb → book

Nominal → book | flight

NP → Aberdeen

Det → the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

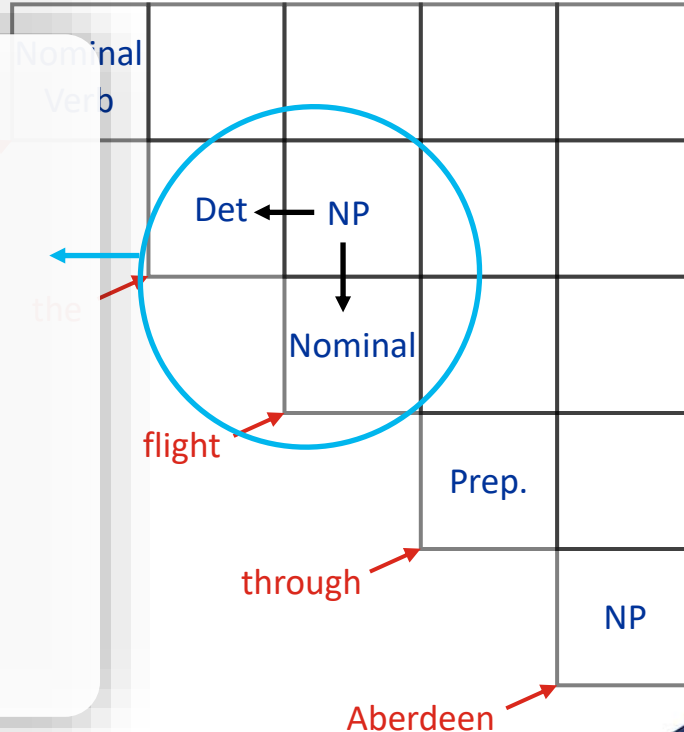
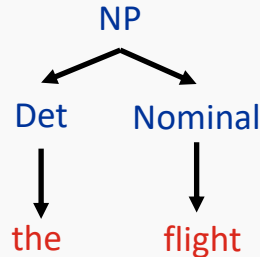
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

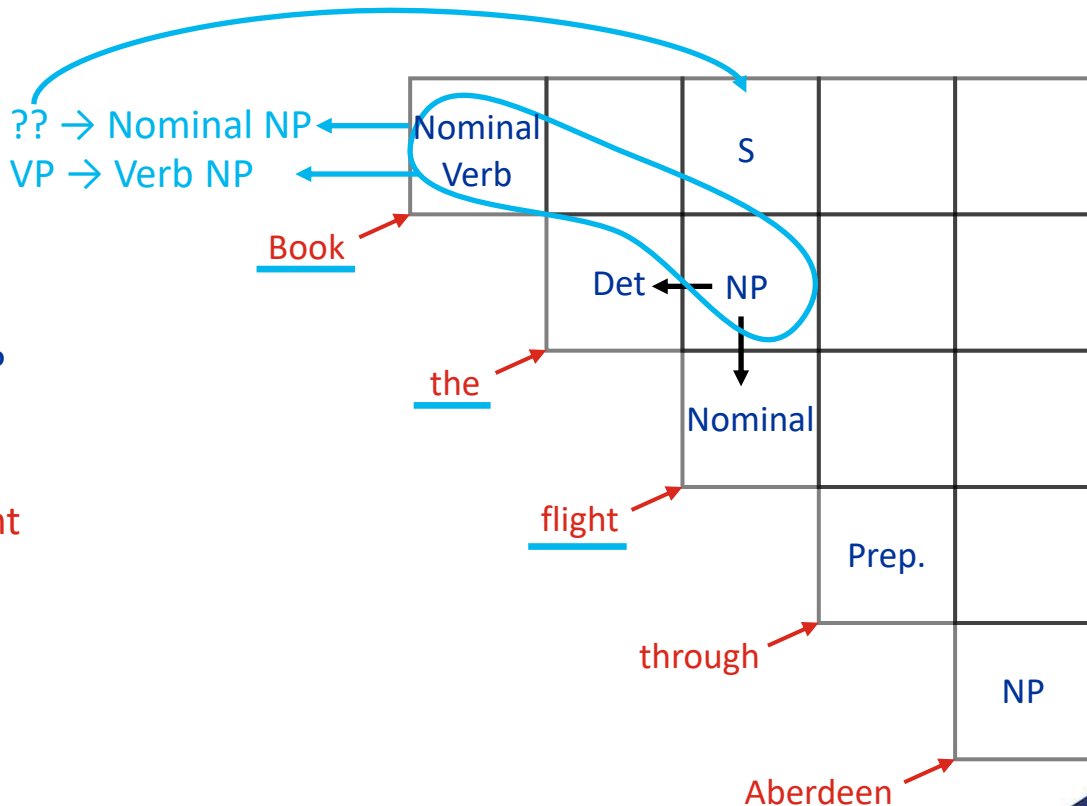
Verb → **book**

Nominal → **book** | **flight**

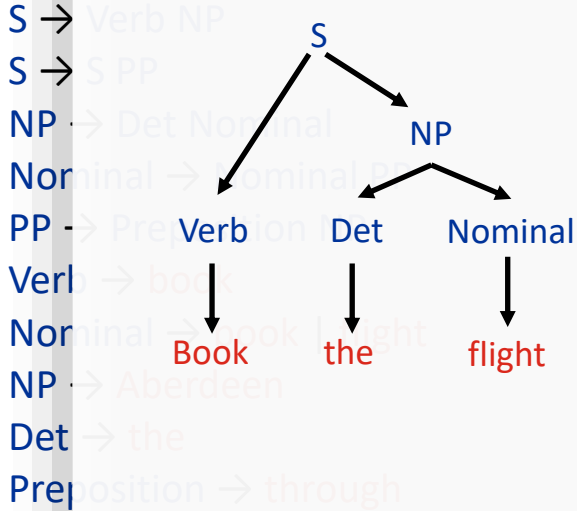
NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)



Book

Nominal
Verb

S

Det

NP

the

Nominal

flight

Prep.

through

NP

Aberdeen

CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$$S \rightarrow S \text{ PP}$$

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

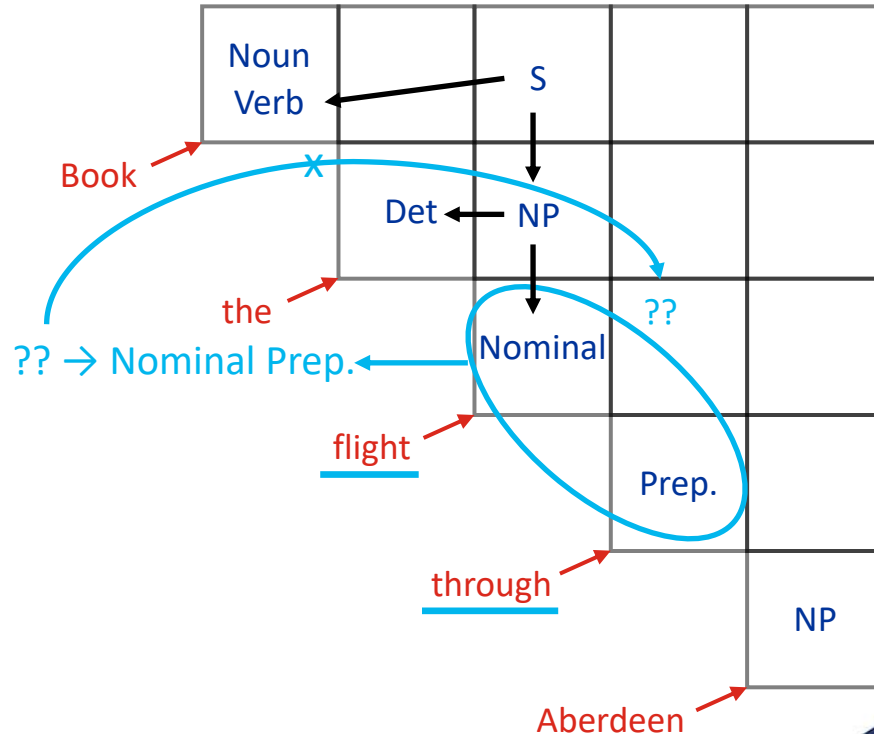
Verb → book

Nominal $\rightarrow$ book | flight

NP → Aberdeen

Det $\rightarrow$ the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

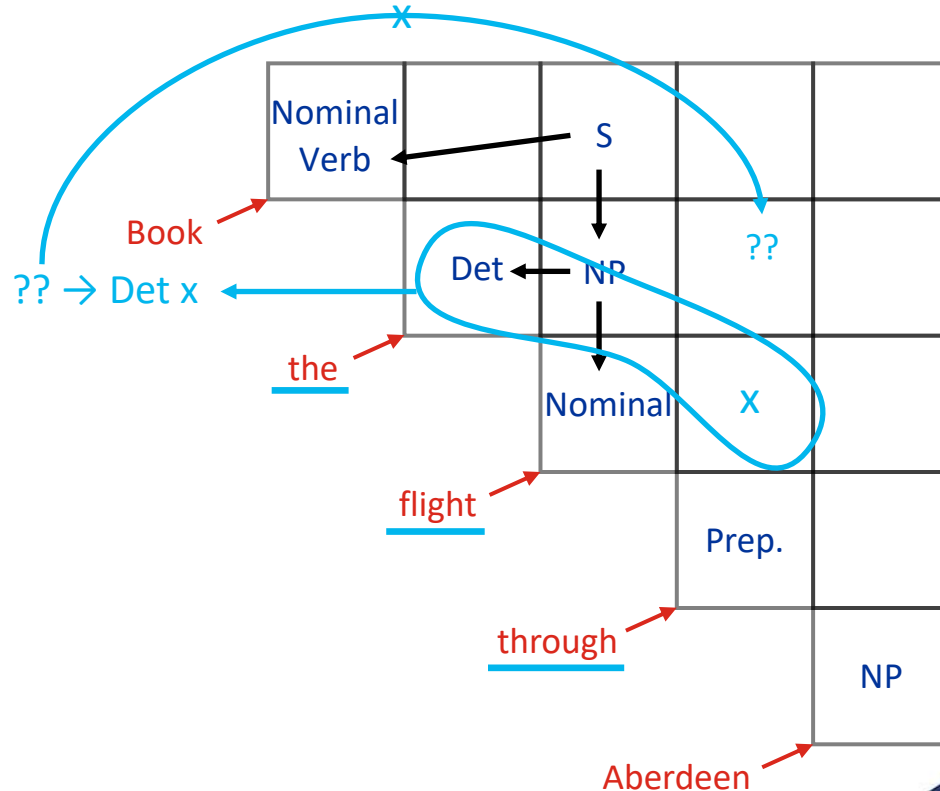
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$$S \rightarrow S PP$$

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

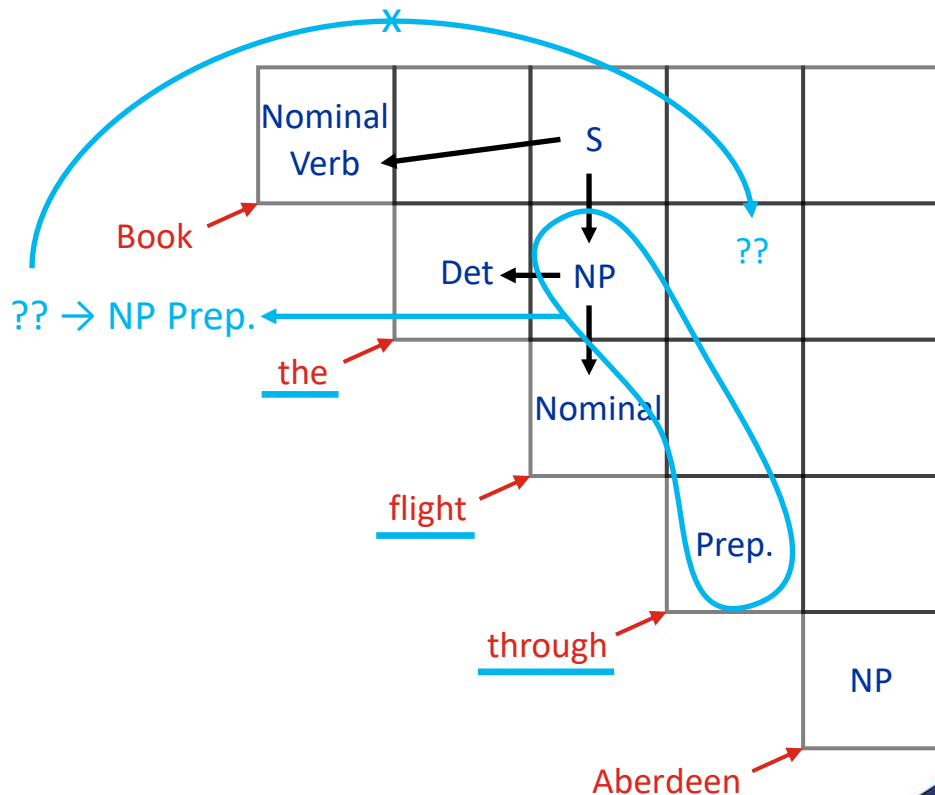
Verb → book

Nominal $\rightarrow$ book | flight

NP → Aberdeen

Det $\rightarrow$ the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

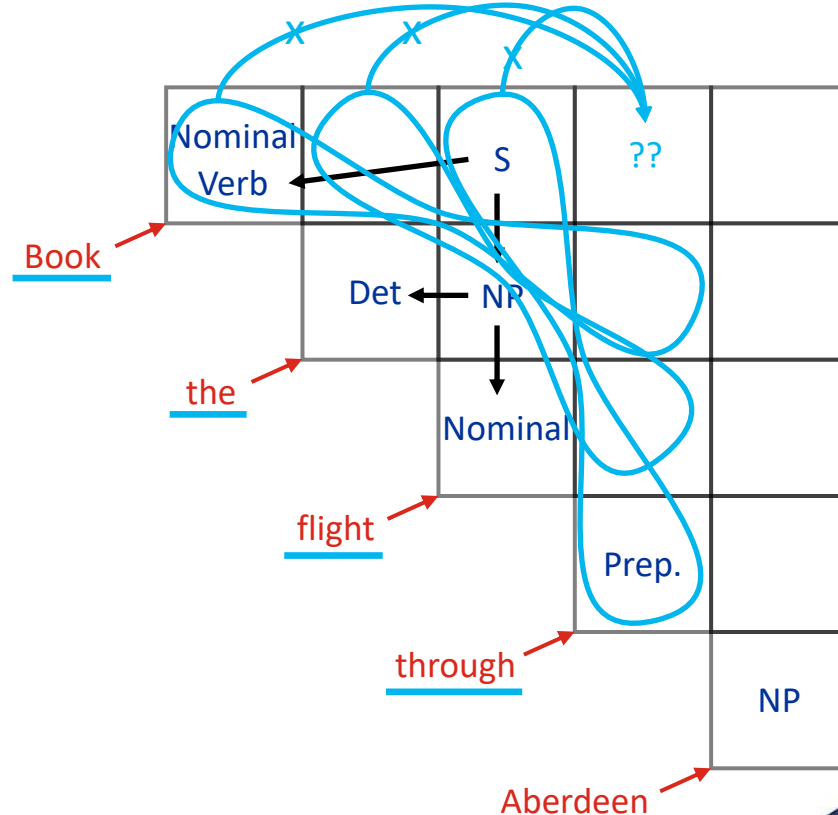
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

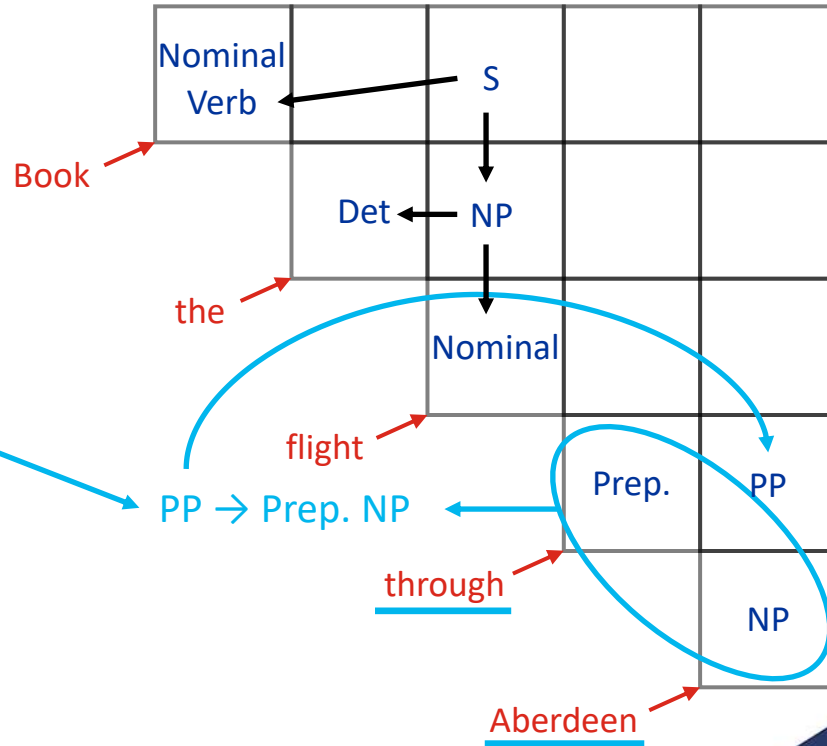
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$$S \rightarrow S \text{ PP}$$

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition N

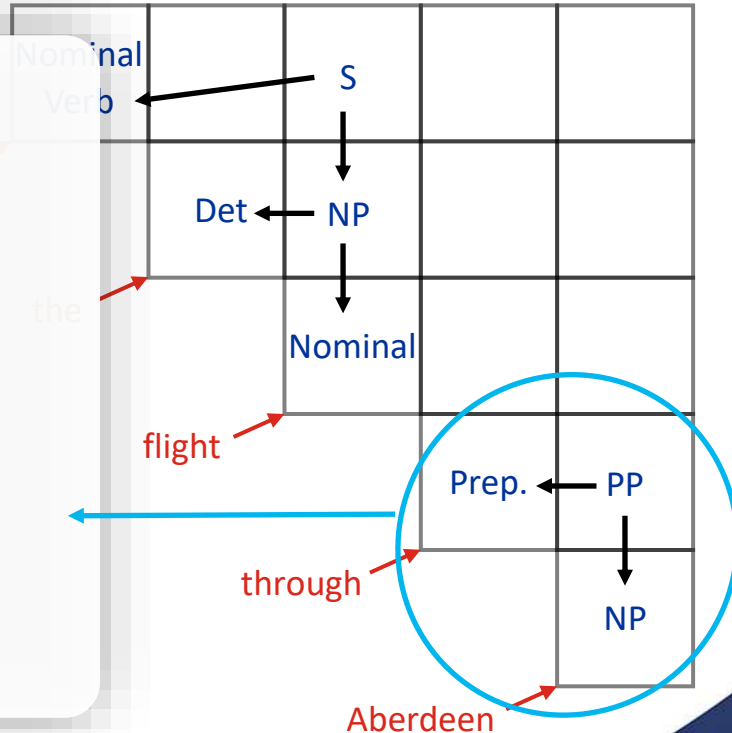
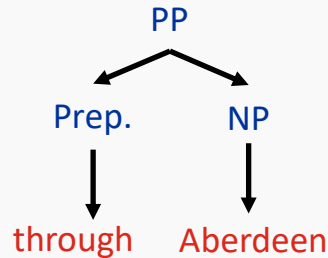
Verb → book

Nominal $\rightarrow$ book | flight

NP → Aberdeen

Det → the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

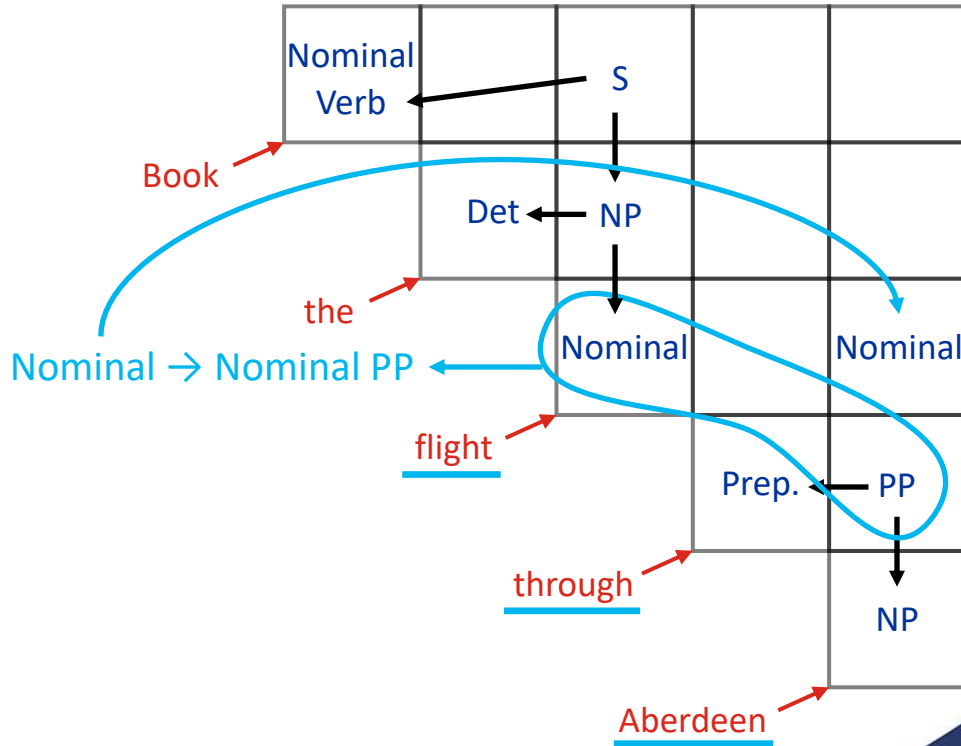
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

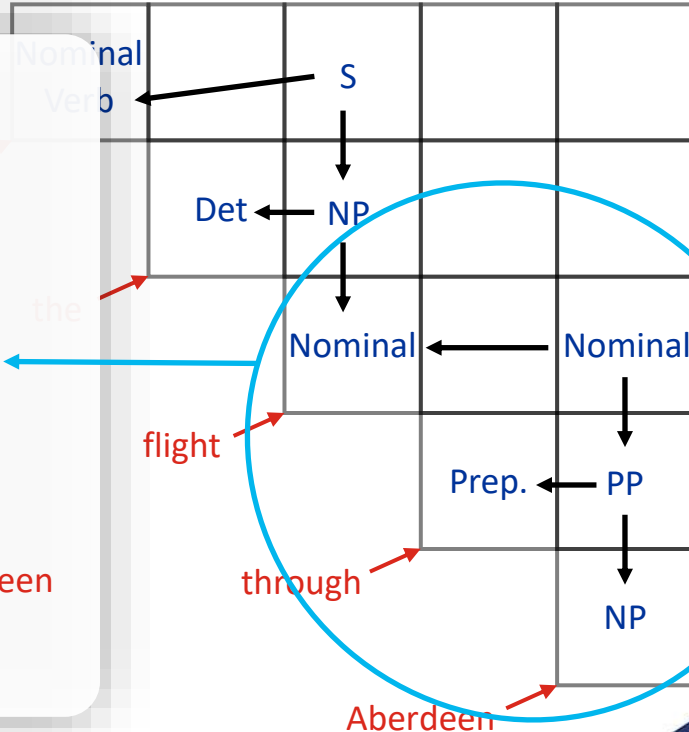
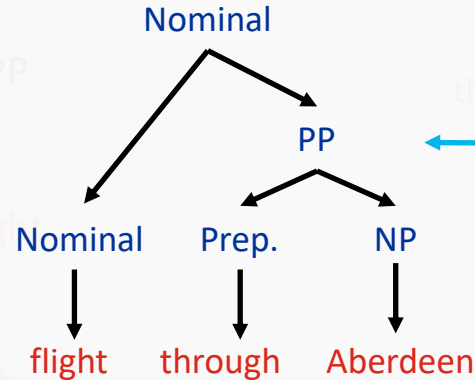
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$$S \rightarrow S \text{ PP}$$

NP → Det Nominal → NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

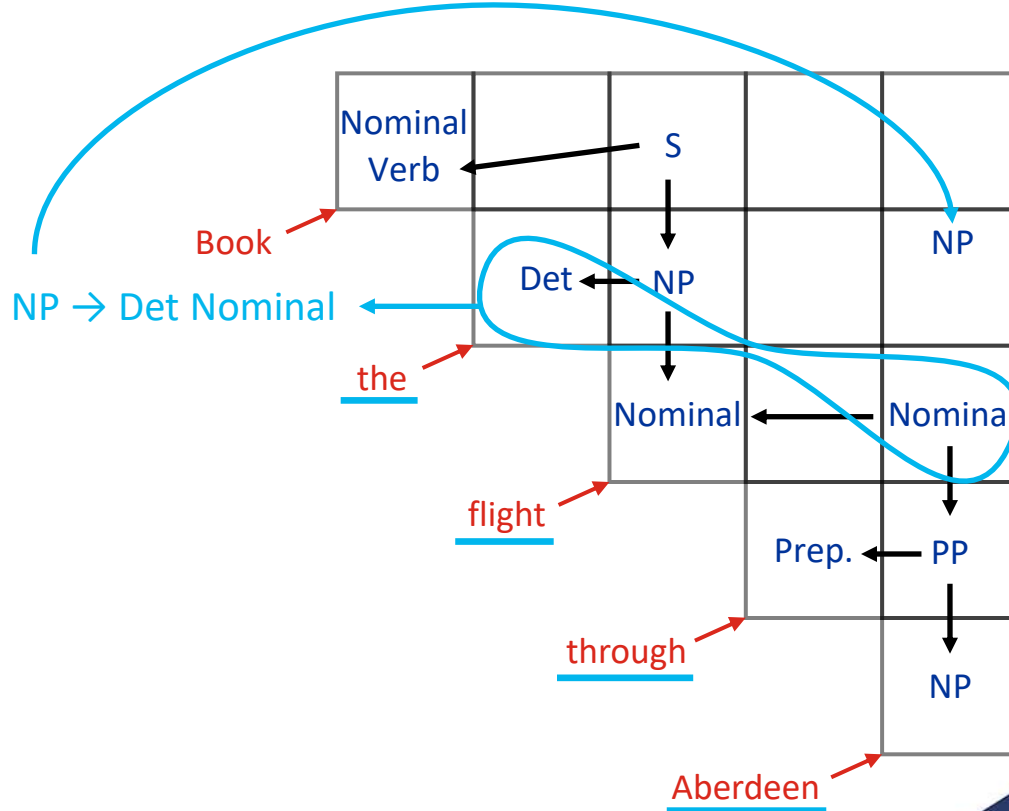
Verb → book

Nominal $\rightarrow$ book | flight

NP → Aberdeen

Det → the

Preposition → through



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

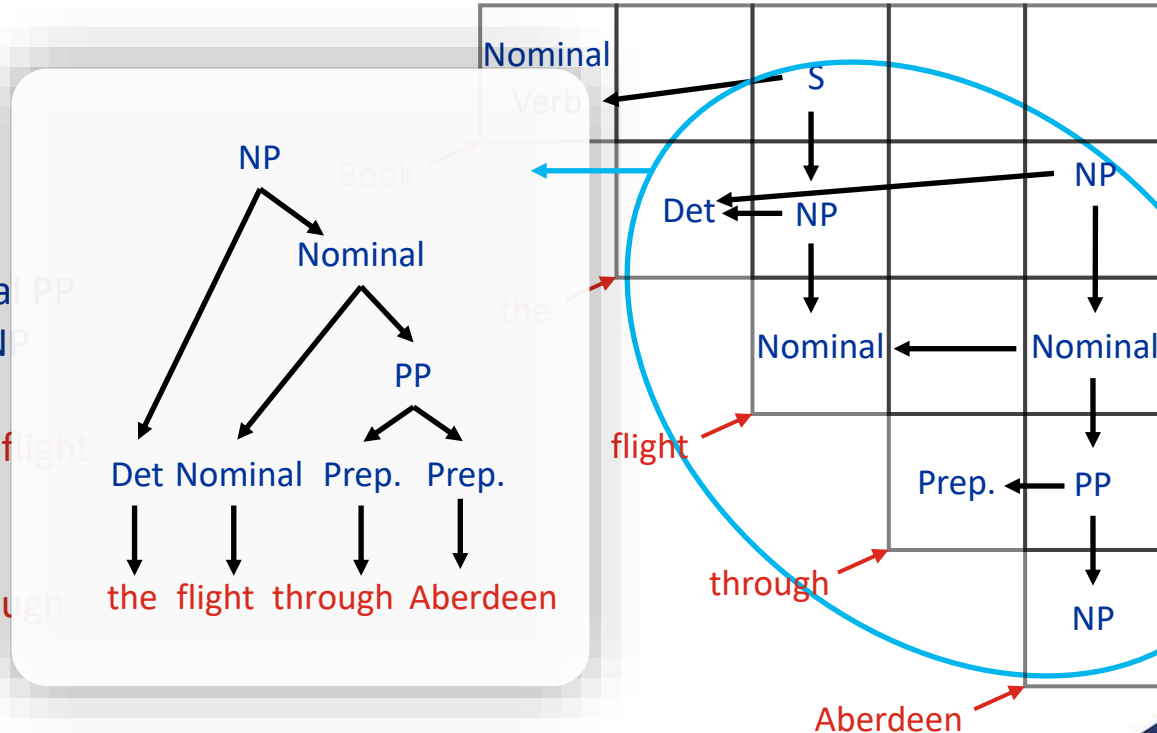
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

?? → Nominal NP
S → Verb NP

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

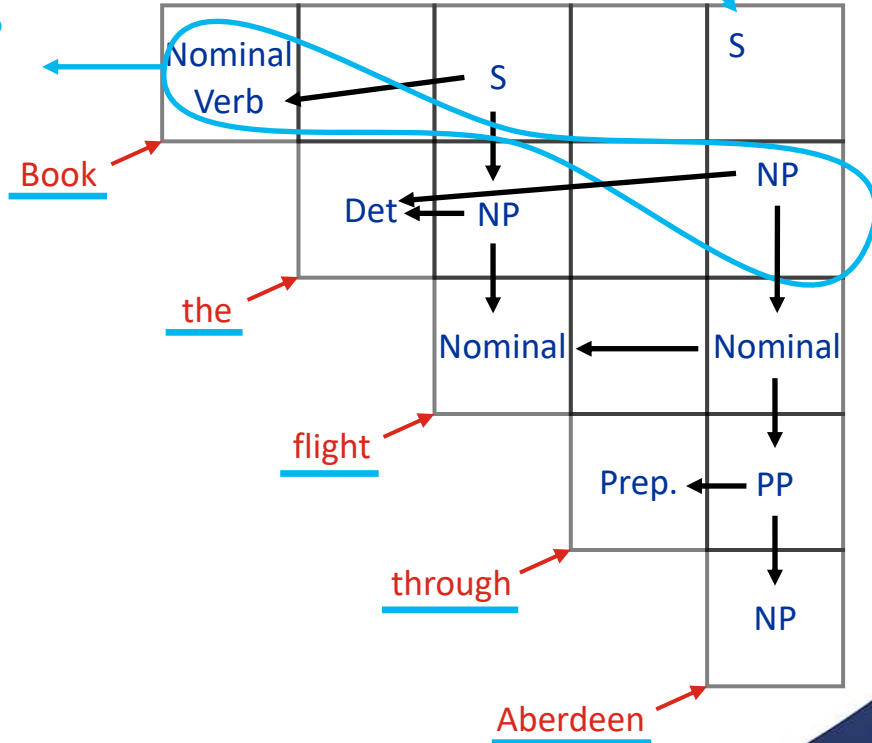
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

S → Verb NP

S → S PP

NP → Det Nominal

Nominal → Nominal PP

PP → Preposition NP

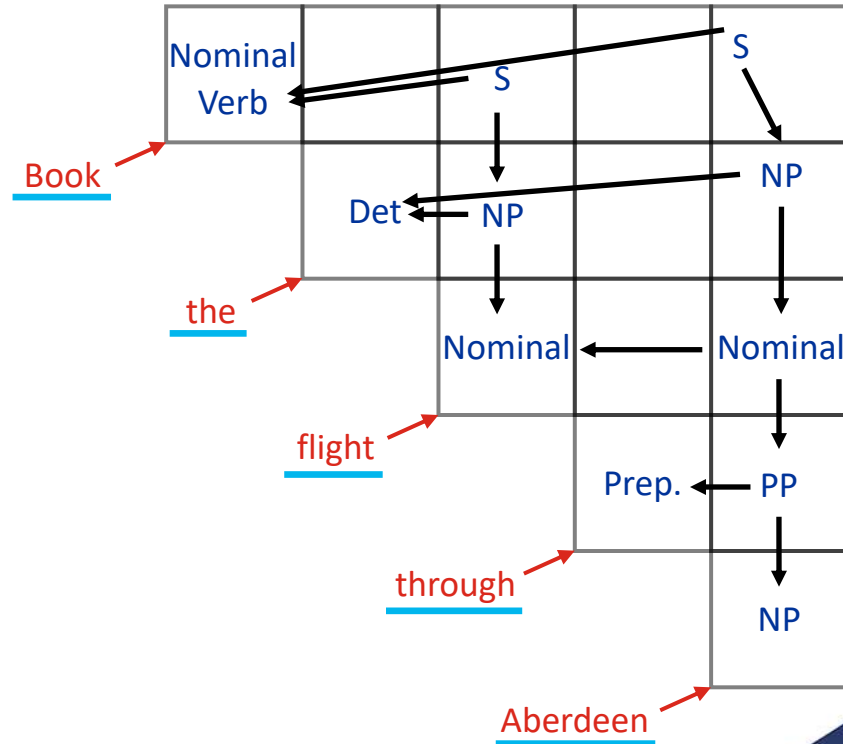
Verb → **book**

Nominal → **book** | **flight**

NP → **Aberdeen**

Det → **the**

Preposition → **through**



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$S \rightarrow S \text{ PP}$

$\text{NP} \rightarrow \text{Det Nominal}$

$\text{Nominal} \rightarrow \text{Nominal PP}$

$\text{PP} \rightarrow \text{Preposition NP}$

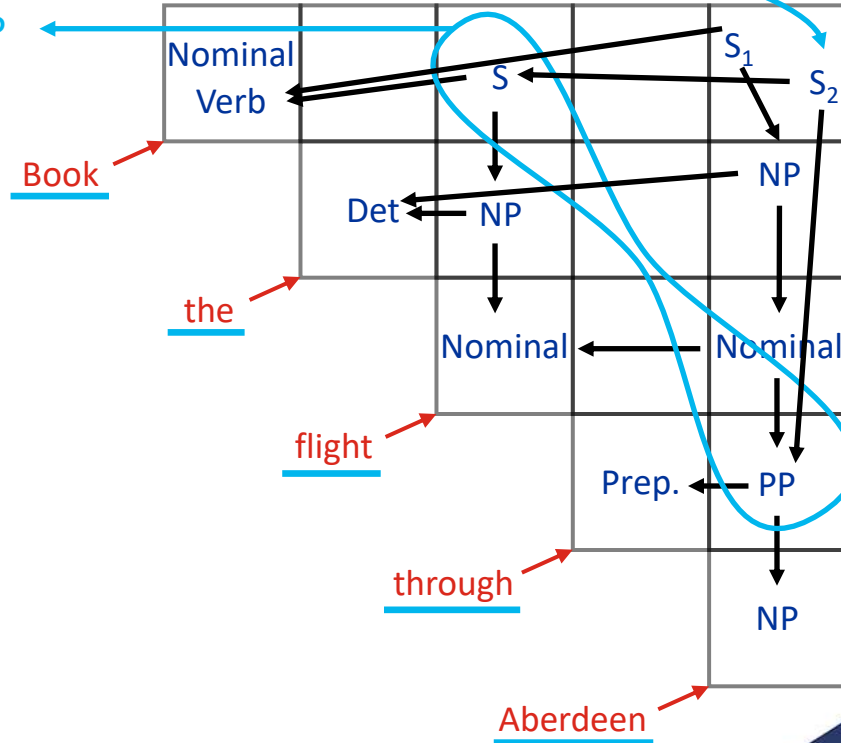
$\text{Verb} \rightarrow \text{book}$

$\text{Nominal} \rightarrow \text{book} \mid \text{flight}$

$\text{NP} \rightarrow \text{Aberdeen}$

$\text{Det} \rightarrow \text{the}$

$\text{Preposition} \rightarrow \text{through}$



CKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP}$

$S \rightarrow S \text{ PP}$

$\text{NP} \rightarrow \text{Det Nominal}$

$\text{Nominal} \rightarrow \text{Nominal PP}$

$\text{PP} \rightarrow \text{Preposition NP}$

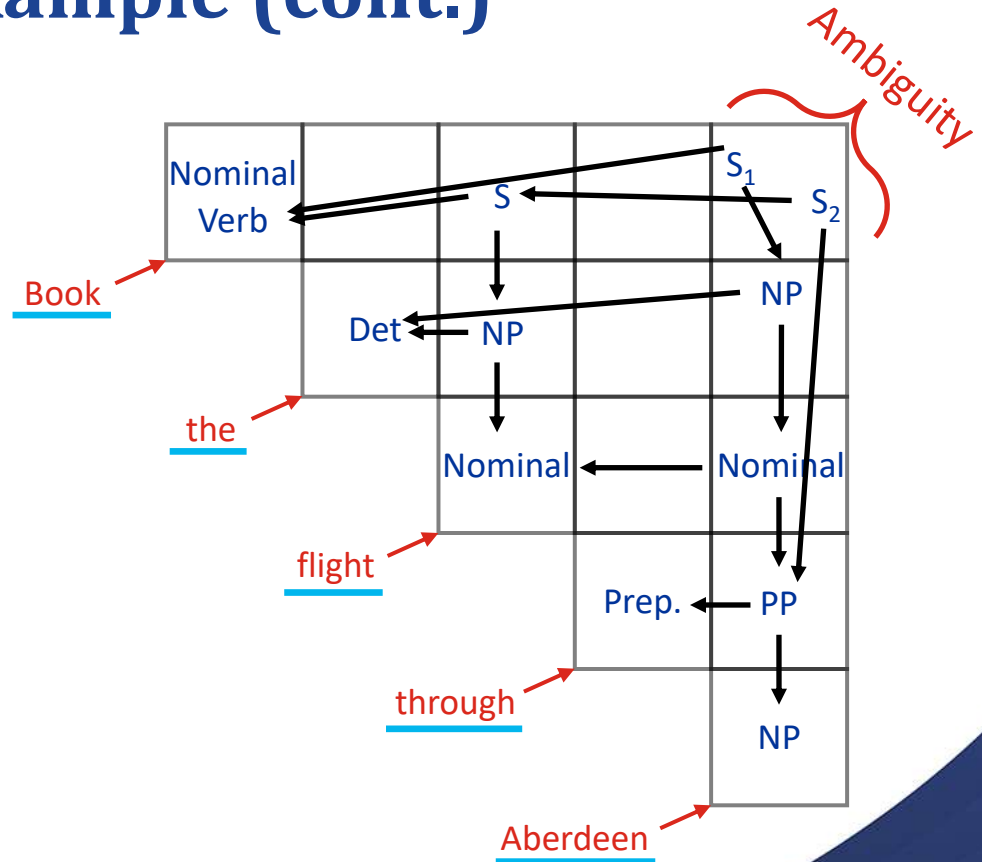
$\text{Verb} \rightarrow \text{book}$

$\text{Nominal} \rightarrow \text{book} \mid \text{flight}$

$\text{NP} \rightarrow \text{Aberdeen}$

$\text{Det} \rightarrow \text{the}$

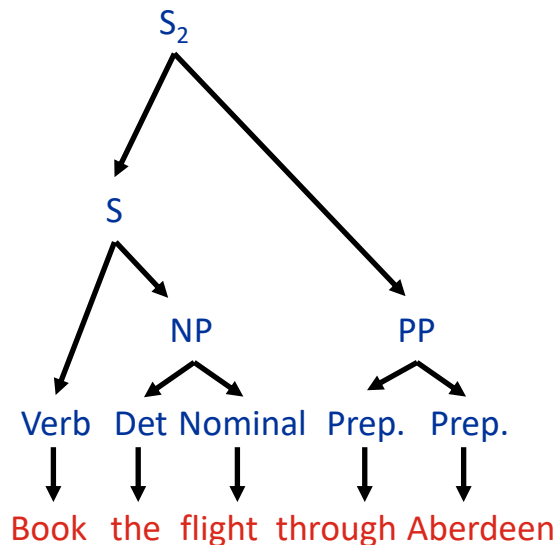
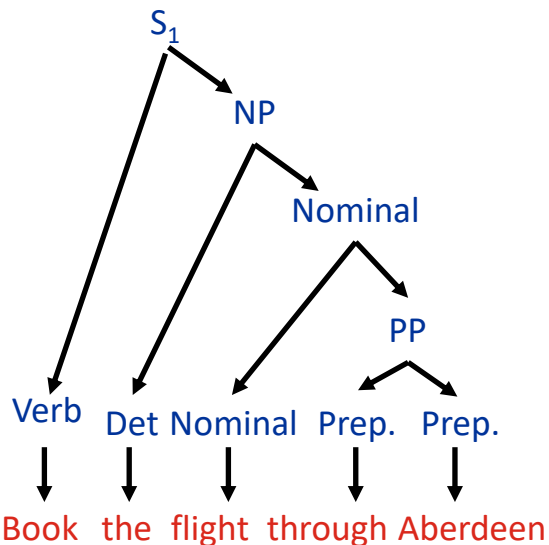
$\text{Preposition} \rightarrow \text{through}$



CKY Algorithm Example (cont.)

Now, we obtained the parse trees for the sentence "Book the flight through Aberdeen." Although most sentences have a unique parse tree, this example yielded two, highlighting the sentence's inherent ambiguity.

Additional methods are required to determine the final result.



CKY Algorithm Example (cont.)

The overall CKY algorithm is given by the following pseudocode.

```
function CKY-PARSE(words, grammar) returns table  
  
  for  $j \leftarrow$  from 1 to LENGTH(words) do  
    for all  $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$   
       $\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$   
    for  $i \leftarrow$  from  $j-2$  down to 0 do  
      for  $k \leftarrow i+1$  to  $j-1$  do  
        for all  $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C \in \text{table}[k, j]\}$   
           $\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$ 
```

Probabilistic CKY Parsing

Probabilistic CKY Parsing

Probabilistic CKY Parsing is an extension of the standard CKY parsing algorithm that incorporates probabilistic information.

It is used to not only determine if a sentence can be generated by a grammar but also to find the most likely parse tree for that sentence based on probabilities assigned to the grammar's rules.

This approach is particularly useful in dealing with ambiguities in natural language, where multiple parse trees might be possible for a single sentence.

Probabilistic Context-Free Grammar

In a Probabilistic Context-Free Grammar (PCFG), each production rule has an associated probability that indicates how likely the rule is to be used in generating a sentence. E.g.:

CFG: $S \rightarrow \text{Verb NP}$, $S \rightarrow \text{Verb NP PP}$

PCFG: $S \rightarrow \text{Verb NP}$ (0.6), $S \rightarrow \text{Verb NP PP}$ (0.4)

The probabilities of all rules expanding from a given non-terminal sum to 1.

Probabilistic Context-Free Grammar

The example grammar with estimated probabilities.

Verb $\rightarrow$ **book** (1.0)
Noun $\rightarrow$ **book** (5.0) | **flight** (5.0)
Proper-Noun $\rightarrow$ **Aberdeen** (1.0)
Det $\rightarrow$ **the** (1.0)
Preposition $\rightarrow$ **through** (1.0)

CFG Lexicon

$S \rightarrow VP$ (1.0)
 $VP \rightarrow Verb\ NP$ (0.6)
 $VP \rightarrow Verb\ NP\ PP$ (0.4)
 $NP \rightarrow Det\ Nominal$ (0.7)
 $NP \rightarrow Proper-Noun$ (0.3)
 $Nominal \rightarrow Noun$ (0.8)
 $Nominal \rightarrow Nominal\ PP$ (0.2)
 $PP \rightarrow Preposition\ NP$ (1.0)

CFG Syntax

Conversion to CNF with Probabilities

When converting to CNF, the probabilities from the PCFG can be inherited.

Verb → book (1.0)
~~Nominal~~ → book (0.8x0.5) | flight
(0.8x0.5)
NP → Aberdeen (0.7x1.0)
Det → the (1.0)
Preposition → through (1.0)

CNF Lexicon

~~S~~ → ~~VP~~
S → Verb NP (0.6)
S → S PP (0.4)
NP → Det Nominal (0.7)
~~NP~~ → ~~Proper Noun~~
~~Nominal~~ → ~~Noun~~
Nominal → Nominal PP (0.2)
PP → Preposition NP (1.0)

CNF Syntax

PCKY Algorithm Example (cont.)

S → Verb NP (0.6)

S → S PP (0.4)

NP → Det Nominal (0.7)

Nominal → Nominal PP (0.2)

PP → Preposition NP (1.0)

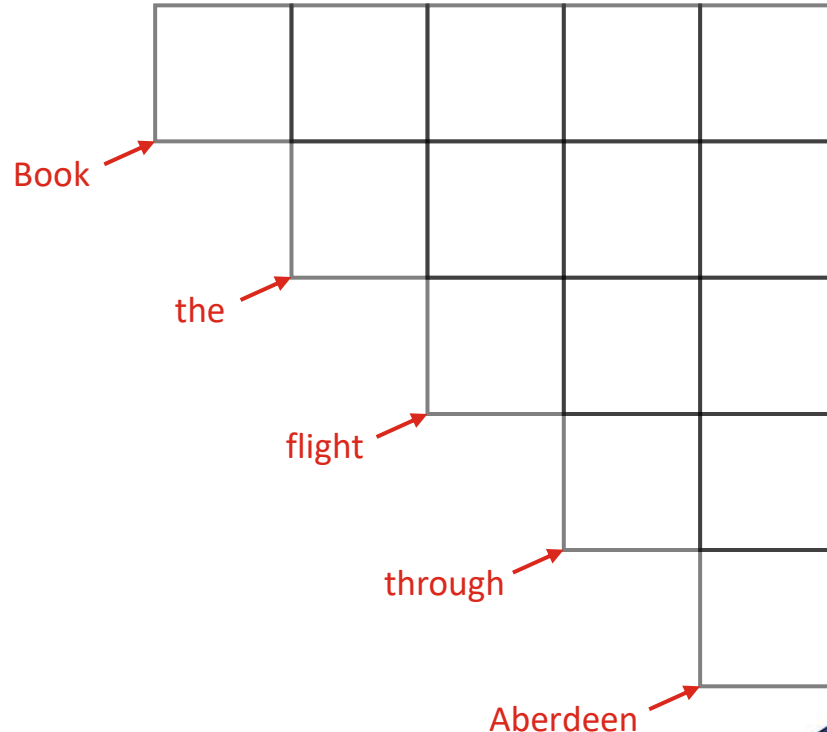
Verb → book (1.0)

Nominal → book (0.4) | flight (0.4)

NP → Aberdeen (0.7)

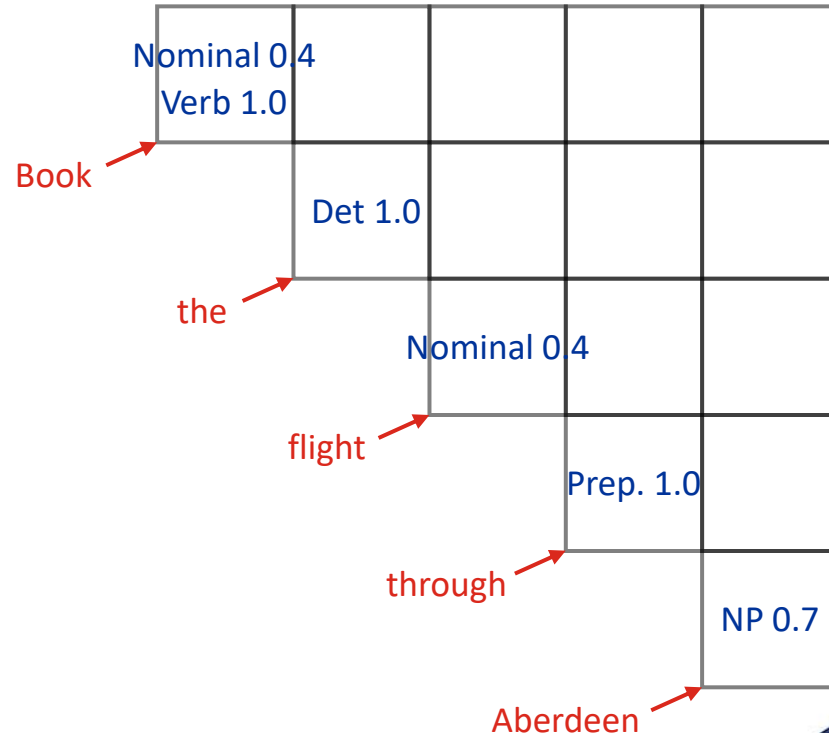
Det → the (1.0)

Preposition → through (1.0)



PCKY Algorithm Example (cont.)

S → Verb NP (0.6)
S → S PP (0.4)
NP → Det Nominal (0.7)
Nominal → Nominal PP (0.2)
PP → Preposition NP (1.0)
Verb → **book** (1.0)
Nominal → **book** (0.4) | **flight** (0.4)
NP → **Aberdeen** (0.7)
Det → **the** (1.0)
Preposition → **through** (1.0)



PCKY Algorithm Example (cont.)

$S \rightarrow \text{Verb NP} \quad (0.6)$

$$S \rightarrow SPP \quad (0.4)$$

NP → Det Nominal (0.7)

Nominal → Nominal PP (0.2)

PP → Preposition NP (1.0)

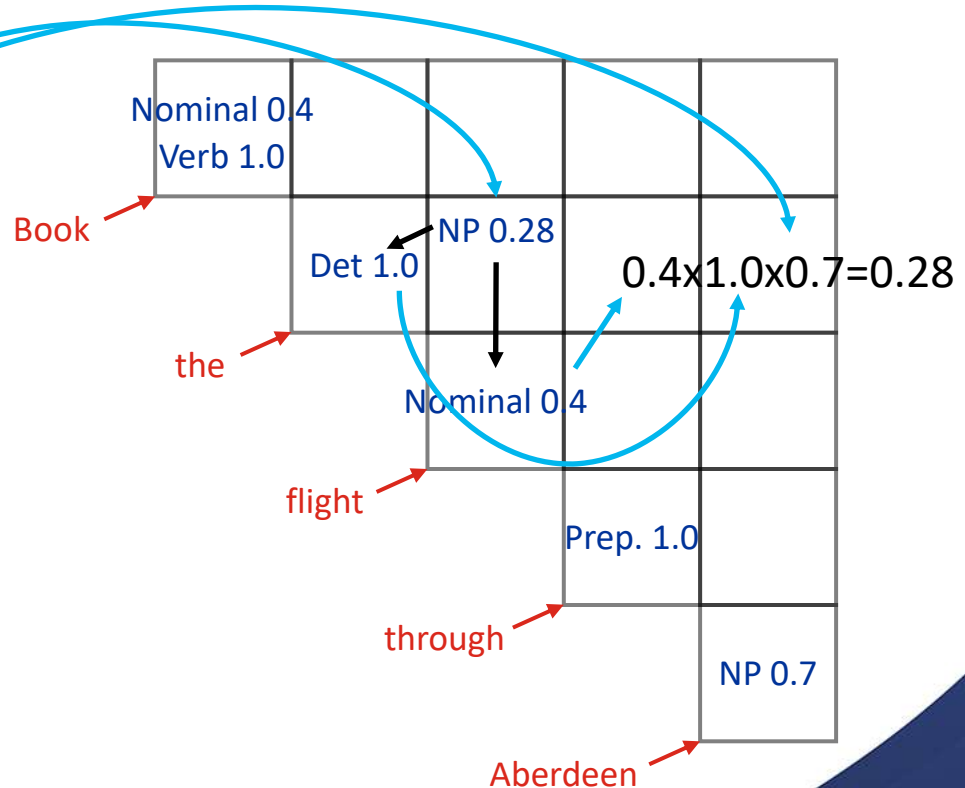
Verb → book (1.0)

Nominal $\rightarrow$ book (0.4) | flight (0.4)

NP → Aberdeen (0.7)

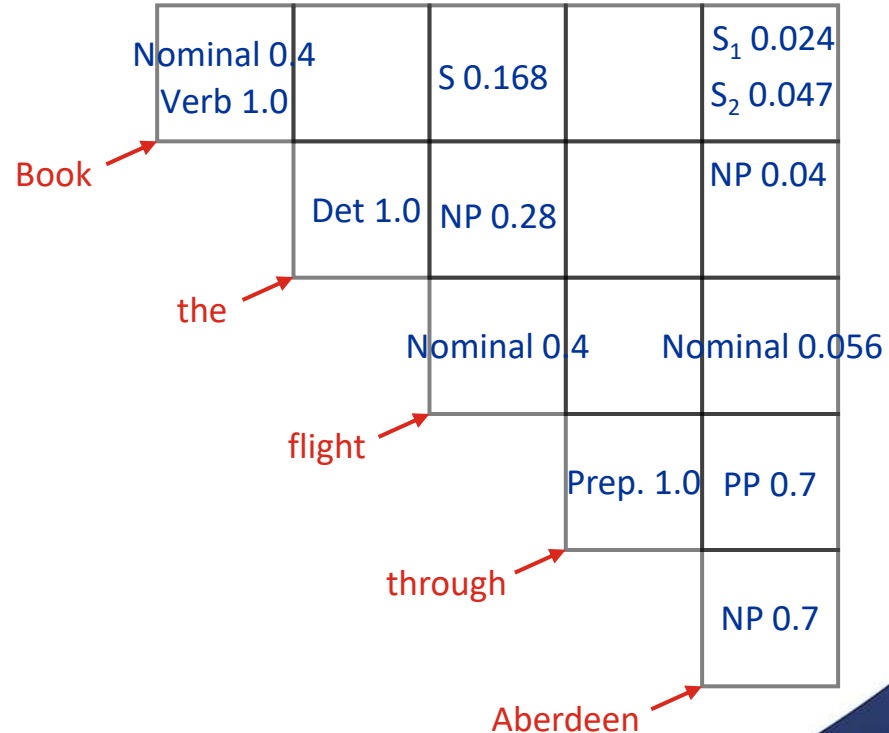
Det → the (1.0)

Preposition → through (1.0)



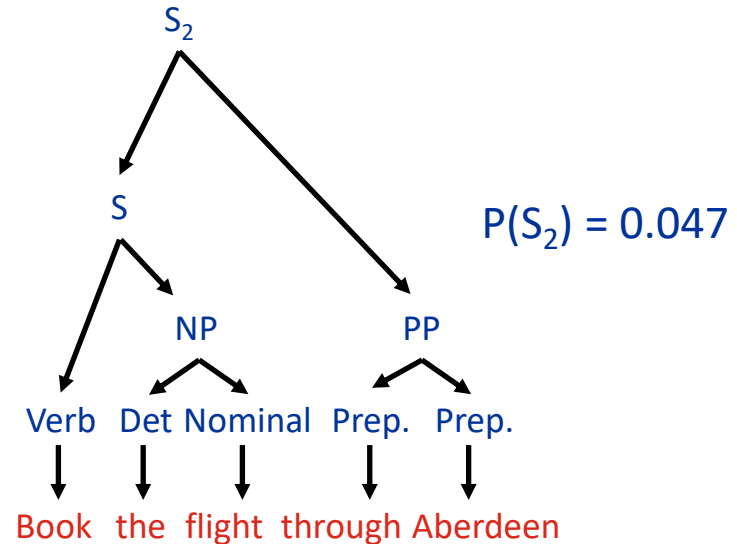
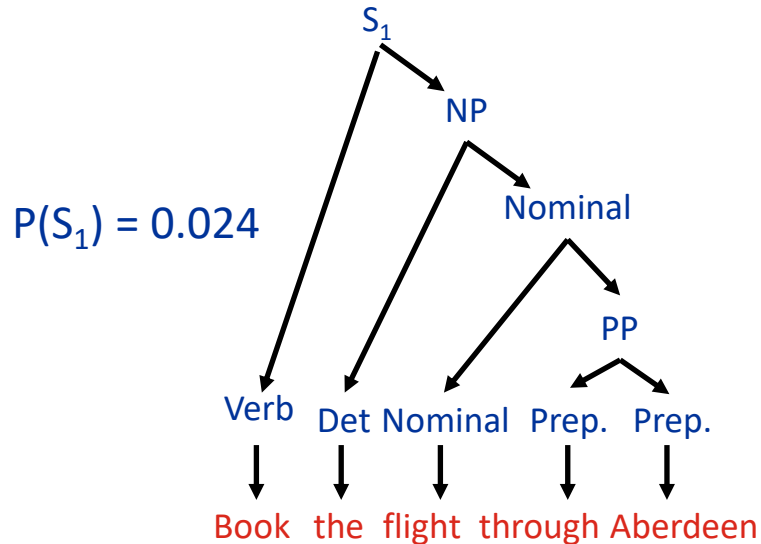
PCKY Algorithm Example (cont.)

S → Verb NP (0.6)
 S → S PP (0.4)
 NP → Det Nominal (0.7)
 Nominal → Nominal PP (0.2)
 PP → Preposition NP (1.0)
 Verb → **book** (1.0)
 Nominal → **book** (0.4) | **flight** (0.4)
 NP → **Aberdeen** (0.7)
 Det → **the** (1.0)
 Preposition → **through** (1.0)



PCKY Algorithm Example (cont.)

According to the probabilities, S_2 is more plausible.



Limitation of CFG/PCFG

Using context-free grammars (CFGs) to analyse natural language has several straightforward limitations

Inability to Capture Contextual Dependencies

CFGs are "context-free," meaning they cannot account for dependencies between words or phrases that depend on the broader context of the sentence.

For example, subject-verb agreement, where the verb form must agree with the subject in number and person, cannot be effectively handled by CFGs because they lack the mechanism to refer back to previously parsed elements.

Limited Expressiveness

CFGs are limited in their ability to describe natural languages fully.

Human languages often exhibit patterns and rules that go beyond the capabilities of CFGs, such as nested dependencies and certain forms of syntactic ambiguity that CFGs cannot resolve.

This limitation leads to an inability to generate accurate parse trees for complex sentences.

Ambiguity Handling

CFGs often generate multiple parse trees for sentences with ambiguous syntax (i.e., sentences that can be interpreted in more than one way).

While ambiguity is a natural aspect of human languages, CFGs do not have a built-in mechanism to prefer one interpretation over another, requiring additional disambiguation processes or probabilistic approaches.

Lack of Lexical Sensitivity

CFGs typically treat words as atomic units and do not account for variations in meaning or usage based on lexical context.

This limitation can lead to incorrect parsing of sentences where the meaning of a word depends on its context, a common occurrence in natural language.

Inflexibility with Idiomatic Expressions

CFGs struggle with idiomatic expressions and phrases that do not conform to standard grammatical rules.

Such expressions often require special handling that CFGs are not inherently equipped to provide, as they rely on fixed, non-compositional meanings that CFGs cannot easily capture.

Scalability Issues

As CFGs grow to cover more of a language's grammar, they can become very large and complex, making them difficult to manage and process.

This scalability issue becomes particularly problematic when trying to cover the vast and nuanced range of constructions present in any natural language.

Inadequate for Parsing Non-Syntactic Elements

CFGs focus on syntactic structure and are not equipped to handle non-syntactic elements of language, such as pragmatics, discourse structure, and prosody, which often play a crucial role in understanding natural language.

Conclusion

Understanding CFGs: Explained the concept of context-free grammars and their role in defining sentence structure through non-terminals, terminals, and production rules.

Parse Trees: Demonstrated how parse trees visualize the hierarchical structure of sentences, helping identify grammatical relationships.

CKY Parsing Algorithm: Introduced the CKY algorithm for efficiently parsing sentences, highlighting its application in CFG parsing.

Ambiguity and PCFG: Addressed the limitations of CFGs in handling syntactic ambiguity and the role of probabilistic CFGs in resolving ambiguities.

Limitations of CFGs: Acknowledged that CFGs cannot handle contextual dependencies, lexical sensitivity, or idiomatic expressions effectively.